CloneCalls

Security Testing Analysis & Final Report

Deliverable 3

1. Executive Summary

CloneCalls is an AI-powered voice clone application built with FastAPI, Supabase, Pinecone, OpenAI, and ElevenLabs. This report covers the complete security testing lifecycle of the implementation: the threat model, all security controls, dynamic security testing results using Postman and OWASP ZAP, a structured code review, and the final security fixes applied as a result of testing.

The project began with 14 base security controls. A three-phase testing programme — static code review, dynamic API testing (Postman), and automated scanning (OWASP ZAP) — identified 10 vulnerabilities and misconfigurations. All 10 were resolved. The final system achieves 25/25 Postman tests passing and 2 residual ZAP alerts, both of which are documented architectural constraints of a local HTTP environment.

Postman Tests	ZAP Alerts Fixed	Vulnerabilities Fixed
25/25	**9/11**	**10**
100% passing	2 env. limitations	found and resolved

2. Threat Model

2.1 System Assets

Asset	Description	Risk if Compromised
User credentials	Email addresses and hashed passwords stored in Supabase	Account takeover, identity theft
JWT access tokens	Bearer tokens used to authenticate all protected API calls	Full user impersonation
Briefing memories	Personal daily context stored as vectors in Pinecone	Privacy violation, data leak
Voice audio	Raw .webm recordings uploaded by callers	Sensitive audio accumulates on server
API keys	Supabase, OpenAI, Pinecone, ElevenLabs credentials	Unauthorised API usage, billing fraud

2.2 STRIDE Threat Analysis

Threat	Category	Attack Scenario	Control
Token forgery	Spoofing	Attacker crafts fake JWT to impersonate a user	Server-side Supabase token validation on every request
Cross-user data access	Tampering	User changes target_email to access another user's briefings	Triple-filter Pinecone queries (owner + caller + date)
PII leakage	Info Disclosure	User accidentally includes SSN or credit card in briefing	Regex-based PII scrubbing before any storage
Prompt injection	Tampering	Caller manipulates AI into revealing out-of-scope data	Constrained system prompt with CRITICAL SECURITY RULE
Memory exhaustion DoS	Denial of Service	Attacker uploads 500 MB audio file to crash the server	10 MB file size cap — rejected before memory allocation
API enumeration	Info Disclosure	Attacker browses Swagger UI to map all endpoints	openapi_url=None; docs disabled by default
Credential brute force	Elev. of Privilege	Attacker tries thousands of passwords against /login	Generic error messages; Supabase account lockout
Clickjacking	Tampering	App embedded in iframe to hijack user clicks	X-Frame-Options: DENY + CSP frame-ancestors 'none'
MIME sniffing	Tampering	Browser misinterprets API response as executable content	X-Content-Type-Options: nosniff
XSS via error messages	Injection	Attacker injects HTML via API error strings in UI	showMsg() rewritten to use textContent

Hung connection DoS	Denial of Service	Slow ElevenLabs response stalls a server thread permanently	30-second timeout on all external API calls
Key exposure in logs	Info Disclosure	API keys appear in server access logs via URL params	All keys passed in HTTP headers only

3. Security Features Implemented

#	**Feature**	**File**	**Threat Addressed**
1	JWT server-side validation via Supabase	security.py	Token forgery, unauthenticated access
2	Password min 8 / max 50 chars via Pydantic	schemas.py	Trivial password brute force
3	Triple-filter Pinecone queries	routes.py	IDOR, horizontal privilege escalation
4	Server-side identity enforcement from token	routes.py	Privilege escalation, IDOR
5	Pydantic schema validation on all body fields	schemas.py	Malformed input, type injection
6	Query(pattern=...) on email query parameters	routes.py	Parameter tampering
7	10 MB audio file size cap with 413 response	routes.py	Memory exhaustion DoS
8	Empty audio file check — 422 not 500	routes.py	Unhandled exception, info leakage
9	Temporary audio file deletion in finally block	routes.py	Sensitive data accumulation on filesystem
10	Regex PII scrubbing before Pinecone write	security.py	PII exposure in third-party vector database
11	Constrained AI system prompt	routes.py	Prompt injection, AI jailbreaking

12	CORS strict origin allowlist from .env	main.py	CSRF, cross-origin API access
13	Security response headers middleware (6 headers)	main.py	Clickjacking, MIME sniffing, XSS
14	Environment variables + fail-fast validation	config.py	Credential exposure in source code
15	Generic error messages on login and signup	routes.py	Email enumeration, info disclosure
16	API docs disabled (openapi_url=None)	main.py	Endpoint enumeration via Swagger UI
17	30-second timeout on ElevenLabs calls	routes.py	DoS via hung external connections
18	API keys in headers not URL parameters	routes.py	Key exposure in server access logs
19	showMsg() uses textContent not innerHTML	index.html	XSS via error message injection
20	JWT stored and cleared correctly in localStorage	index.html	Session fixation, orphaned tokens
21	Password visibility toggle with aria-label	index.html	Shoulder-surfing
22	CSP meta tag in frontend HTML	index.html	Script injection in browser context

4. Secure Code Review

A manual line-by-line review of all five source files was conducted before running any dynamic tests. Findings are classified by severity. This technique is most effective at finding logic errors, missing validations, and insecure patterns that runtime tools cannot detect.

4.1 routes.py

Finding	Severity	Status

voice_call accepted any string for target_email Form field — Query(pattern=...) cannot be applied to Form params	**High**	Fixed — manual _EMAIL_RE.match() validation added
Empty audio file caused unhandled exception — returned 500 response	**High**	Fixed — len(audio_bytes) == 0 check added, returns 422
/openapi.json returned full API schema publicly	**Medium**	Fixed — openapi_url=None when ENABLE_DOCS=false
check_status and simulate_call accepted non-email strings for target_email	**Medium**	Fixed — Query(pattern=r'^[^@\s]+@[^@\s]+\.[^@\s]+$') added
ElevenLabs requests.post() had no timeout	**Medium**	Fixed — timeout=30 added to both outbound calls
ElevenLabs failure returned raw API response body to client	**Medium**	Fixed — generic HTTP 502 raised instead
signup returned raw Supabase exception string	**Medium**	Fixed — replaced with generic error message
signup response included internal user_id field	**Low**	Fixed — user_id removed from response body
print statements logged user email addresses to console (stdout)	**Low**	Fixed — all debug print statements removed

4.2 security.py

Finding	**Severity**	**Status**
PII regex for credit cards uses a loose pattern that could over-redact some non-PII numbers	Low	Accepted — over-redaction (false positives) safer than under-redaction (false negatives)
No PII patterns for email addresses or passport numbers	Low	Accepted risk — out of scope for current implementation phase

4.3 main.py

Finding	Severity	Status
import os appeared after app = FastAPI(...)	Info	Fixed — moved to top of file
ALLOWED_ORIGINS in .env set to placeholder 'https://your-frontend-domain.com'	High	Fixed — updated to correct localhost:5500 and localhost:5501 origins
Security response headers absent from all API responses	Medium	Fixed — SecurityHeadersMiddleware added with 6 headers
openapi_url not disabled alongside docs_url and redoc_url	Medium	Fixed — openapi_url=None added to FastAPI() constructor

4.4 schemas.py

Finding	Severity	Status
full_name field has no whitespace stripping — leading/trailing spaces stored as-is	Low	Documented — low risk, stored in Supabase only, does not affect security
Unused AudioRequest class present in file	Info	Fixed — removed to reduce attack surface understanding confusion

4.5 index.html

Finding	Severity	Status
showMsg() used innerHTML with e.message (user-influenced string from API errors)	High	Fixed — split into showMsg() using textContent and showMsgHTML() for hardcoded HTML only

setLoggedIn() overwrote authToken with localStorage.getItem('cc_token') which was never set	High	Fixed — token now saved to localStorage on login; setLoggedIn() no longer overwrites it
ALLOWED_ORIGINS in .env overrode code default and was set to wrong port	High	Fixed — .env updated to include both ports 5500 and 5501
No session restoration on page reload	Medium	Fixed — IIFE reads localStorage on DOMContentLoaded
Logout did not clear localStorage tokens	Medium	Fixed — localStorage.removeItem() added to logout function
Google Fonts link tag missing crossorigin attribute	Low	Fixed — crossorigin attribute added to all external font links

5. Security Testing Analysis

A three-phase testing programme was used to validate all security controls. Each technique is complementary — static review finds logic errors, dynamic testing validates runtime behaviour, and automated scanning finds configuration and header issues.

5.1 Technique 1 — Static Code Review

Manual line-by-line inspection of all five source files conducted before any dynamic testing. This technique identified 15 issues across 5 files (see Section 4). Static review is uniquely effective at finding logic errors and insecure patterns that runtime tools cannot detect — for example, the setLoggedIn() token overwrite bug where a correct login immediately overwrote the token with null from an empty localStorage read.

Property	**Value**
Scope	All 5 source files: routes.py, security.py, main.py, schemas.py, index.html
Issues found	15 total — 4 High, 6 Medium, 4 Low, 1 Info
Issues resolved	13 resolved; 2 accepted as low-risk or out-of-scope
Unique value	Identifies logic errors, missing validations, and insecure patterns invisible to runtime tools

5.2 Technique 2 — Dynamic API Testing (Postman)

A collection of 25 automated test cases was built and executed against the live running server. Tests covered authentication, input validation, PII scrubbing, information disclosure, CORS behaviour, and the happy path.

Initial run: 22/25 passing. Final run after fixes: 25/25 passing (100%).

#	Test	Category	Initial	Final
1	Valid login — returns 200 + access token	Authentication	**PASS**	**PASS**
2	Wrong password — returns 401	Authentication	**PASS**	**PASS**
3	Login error message is generic (no enumeration)	Authentication	**PASS**	**PASS**
4	No token — returns 401	Authentication	**PASS**	**PASS**
5	Forged token — returns 401	Authentication	**PASS**	**PASS**
6	Password under 8 chars — returns 422	Input Validation	**PASS**	**PASS**
7	Invalid email format on signup — returns 422	Input Validation	**PASS**	**PASS**
8	Briefing text under 5 chars — returns 422	Input Validation	**PASS**	**PASS**
9	Briefing text over 1000 chars — returns 422	Input Validation	**PASS**	**PASS**
10	Invalid allowed_caller email — returns 422	Input Validation	**PASS**	**PASS**
11	Audio file over 10 MB — returns 413	File Upload	**PASS**	**PASS**
12	Empty audio file — returns 422 not 500	File Upload	**FAIL**	**PASS**
13	SSN redacted in stored briefing text	PII Scrubbing	**PASS**	**PASS**
14	Credit card number redacted in stored text	PII Scrubbing	**PASS**	**PASS**

15	Phone number redacted in stored text	PII Scrubbing	**PASS**	**PASS**
16	Raw PII absent from stored text in response	PII Scrubbing	**PASS**	**PASS**
17	/docs endpoint returns 404	Info Disclosure	**PASS**	**PASS**
18	/redoc endpoint returns 404	Info Disclosure	**PASS**	**PASS**
19	/openapi.json returns 404 — not 200	Info Disclosure	**FAIL**	**PASS**
20	Health check returns only {"status":"ok"}	Info Disclosure	**PASS**	**PASS**
21	Signup error message is generic	Info Disclosure	**PASS**	**PASS**
22	Allowed origin receives CORS headers	CORS	**PASS**	**PASS**
23	Unlisted origin receives no CORS headers	CORS	**PASS**	**PASS**
24	Invalid email in check_status — returns 422	Input Validation	**FAIL**	**PASS**
25	Voice call happy path — returns 200	Happy Path	**FAIL**	**PASS**

Bugs found by Postman that required backend fixes:

Test	**Bug Found**	**Fix Applied**
Test 12 — empty audio	Server returned 500 (unhandled exception)	Added len(audio_bytes) == 0 check; now returns 422
Test 19 — /openapi.json	Returned 200 with full API schema	Added openapi_url=None when ENABLE_DOCS=false
Test 24 — invalid email in check_status	Accepted non-email string, returned 200	Added Query(pattern=r'^[^@\s]+@[^@\s]+\.[^@\s]+$') to check_status

5.3 Technique 3 — Automated Security Scanning (OWASP ZAP 2.17.0)

OWASP ZAP was run in Automated Scan mode with the OpenAPI specification imported so all API endpoints were discovered and tested. A full spider, AJAX spider, and active scan were executed against http://127.0.0.1:8000.

Initial scan: 11 alerts. Final scan after fixes: 2 alerts remaining (both documented as non-fixable in a local HTTP environment).

Alert	**Risk**	**Status**	**Resolution**
X-Content-Type-Options Header Missing	Low	FIXED	Added X-Content-Type-Options: nosniff via security headers middleware
Content Security Policy Header Not Set	Medium	FIXED	Added Content-Security-Policy header via middleware and meta tag in frontend
CSP: No Fallback Directive	Medium	FIXED	CSP now includes default-src 'none' as catch-all fallback directive
Missing Anti-Clickjacking Header	Medium	FIXED	Added X-Frame-Options: DENY and frame-ancestors 'none' in CSP
Re-examine Cache-Control Directives	Low	FIXED	Added Cache-Control: no-store, no-cache, must-revalidate to all responses
Sub Resource Integrity Missing	Medium	**PARTIAL**	Added crossorigin attribute to Google Fonts links. Full SRI impossible with Google Fonts dynamic CDN — documented limitation
Cross-Domain Misconfiguration	Medium	FALSE POS.	Flagged URL was firefox.settings.services.mozilla.com (Firefox internal CDN). Our API uses a strict CORS allowlist with no wildcards
Strict-Transport-Security Not Set	Medium	**CANNOT FIX**	HSTS requires HTTPS. Browsers ignore this header over HTTP. Only applicable after deployment with SSL certificate on a real domain

Timestamp Disclosure — Unix	Low	ACCEPTED	Timestamps intentionally returned in briefing responses for user auditability. Values are human-readable dates, not sensitive timestamps
Modern Web Application	Info	N/A	Informational only — ZAP detected JavaScript usage. No action required
Retrieved from Cache	Info	FIXED	Cache-Control headers now prevent future caching of API responses

Notably, OWASP ZAP found zero SQL injection, zero command injection, and zero traditional XSS vulnerabilities — confirming that Pydantic validation and input sanitization are effective against automated attack payloads.

6. Final Security Fixes Applied

The following 10 fixes were applied after testing. None were present in the initial implementation — all were discovered through the testing programme.

#	**Fix Applied**	**File**	**Triggered By**
1	Empty audio file check — returns 422 instead of 500	routes.py	Postman test 12
2	openapi_url=None when docs disabled	main.py	Postman test 19
3	Query(pattern=...) on target_email and message query params	routes.py	Postman test 24
4	Manual _EMAIL_RE.match() validation on voice_call Form field	routes.py	Code review
5	Security headers middleware — 6 headers added	main.py	OWASP ZAP scan
6	showMsg() rewritten to use textContent not innerHTML	index.html	Code review

7	showMsgHTML() created — separate function for hardcoded HTML	index.html	Code review
8	localStorage token lifecycle fixed — save on login, clear on logout	index.html	Code review
9	CORS ALLOWED_ORIGINS fixed from placeholder to correct localhost ports	.env	Manual testing
10	crossorigin attribute added to Google Fonts link tags	index.html	OWASP ZAP scan

7. Unimplemented Controls — Final Justification

The following controls were identified during threat modeling, understood technically, and deliberately excluded. Each exclusion has a specific architectural reason — none are implementation failures. Identifying and justifying these gaps is itself a security engineering practice.

Control	**Group**	**Why Not Implemented**
Multi-Factor Authentication	Domain required	Supabase MFA requires a verified redirect URL (e.g. https://clonecalls.com/auth/callback) — 127.0.0.1 is rejected as a callback target
Email Verification on Signup	Domain required	Supabase verification links need a real registered domain as the callback URL — localhost has no valid destination
Password Reset Flow	Domain required	Same root cause — reset tokens expire unused because they redirect to localhost
OAuth / SSO (Google, GitHub)	Domain required	OAuth providers explicitly reject http://127.0.0.1 as a valid redirect URI for web applications
WebAuthn / Passkeys	Domain required	The WebAuthn spec requires the rpId to be a registered domain — browsers hard-block this on localhost

HTTPS / TLS Encryption	HTTPS required	SSL certificates require a domain name; Let's Encrypt does not issue certificates for IP addresses
HttpOnly Secure Cookies	HTTPS required	The Secure cookie flag is stripped by browsers over HTTP — localStorage is the pragmatic local alternative
HSTS	HTTPS required	Browsers ignore Strict-Transport-Security unless delivered over an active HTTPS connection
Production Rate Limiting	Infrastructure	Application-level rate limiting resets on server restart; real protection requires nginx/Cloudflare with persistent state
JWT Token Revocation	Architecture	JWTs are stateless by design — revocation requires a Redis token blacklist, adding latency to every authenticated request
Persistent Audit Logging	Infrastructure	uvicorn stdout is ephemeral — production logging requires Datadog or CloudWatch with persistent write-only log streams

8. Final Report Summary

8.1 Threat Coverage — STRIDE to Control Mapping

STRIDE Category	**Threats Identified**	**Controls Implemented**	**Coverage**
Spoofing	Token forgery, impersonation	JWT server-side validation, server-side identity enforcement	Full
Tampering	Cross-user data access, prompt injection, clickjacking	Triple-filter queries, constrained prompt, X-Frame-Options	Full
Repudiation	User denies uploading briefing	Supabase auth logs, server-side identity from token	Partial

Information Disclosure	PII leakage, key exposure, API enumeration, XSS via errors	PII scrubbing, headers-only keys, docs disabled, textContent	Full
Denial of Service	Memory exhaustion, hung connections	10 MB cap, 30s timeouts, empty file check	Full
Elevation of Privilege	Credential brute force, bypass allowed_caller	Generic errors, token-enforced identity, triple filter	Full

8.2 OWASP Top 10 Coverage

OWASP Category	**Status**	**Control in CloneCalls**
A01 — Broken Access Control	Mitigated	Triple-filter Pinecone queries; server-side identity from token
A02 — Cryptographic Failures	Partial	JWT via Supabase; HTTPS not possible on localhost (documented)
A03 — Injection	Mitigated	Pydantic validation; PII scrubbing; constrained AI prompt
A04 — Insecure Design	Mitigated	Fail-fast config; constrained prompt; date-scoped memory access
A05 — Security Misconfiguration	Mitigated	Docs disabled; CORS allowlist; 6-header security middleware
A06 — Vulnerable Components	Partial	Dependencies used but no automated pip-audit scan run
A07 — Auth Failures	Mitigated	JWT server-side validation; generic errors; password rules
A08 — Software Integrity Failures	Mitigated	.gitignore for secrets; no hardcoded keys; SRI documented
A09 — Logging and Monitoring	Partial	Basic uvicorn logging; no persistent audit log (env limitation)
A10 — SSRF	N/A	Architecture does not expose an SSRF attack surface

8.3 Testing Results Summary

Testing Technique	Scope	Findings	Resolved	Outcome
Static code review	All 5 source files	15 issues (4H, 6M, 4L, 1I)	13 resolved, 2 accepted	All high-severity issues fixed
Postman dynamic testing	25 API test cases	3 backend bugs	3 resolved	25/25 tests passing
OWASP ZAP automated scan	Full endpoint scan	11 alerts	9 fixed, 2 env. limits	0 high-risk findings

9. Conclusion

CloneCalls implements 22 distinct security controls addressing every threat category identified in the STRIDE model. A three-phase security testing programme static code review, Postman dynamic API testing (25 cases), and OWASP ZAP automated scanning identified 10 vulnerabilities and misconfigurations that were not visible from code inspection alone. All 10 were identified, understood, and resolved.

The final system achieves 25/25 Postman tests passing and 2 residual ZAP alerts, both of which are architectural constraints of a local HTTP development environment rather than implementation defects. OWASP ZAP found zero SQL injection, zero command injection, and zero XSS vulnerabilities in active scanning, validating that input validation and sanitization controls are effective against automated attack payloads.

The system's strongest security property is the triple-filter access control on Pinecone queries: a user cannot access another user's briefing memories even with a valid JWT. They must also be named as the allowed_caller for that specific calendar day. This constitutes multi-dimensional access control that goes well beyond simple authentication.

All unimplemented controls MFA, HTTPS, HttpOnly cookies, email verification, HSTS, OAuth — are infrastructure and deployment concerns, not code concerns. Deployment to a registered domain with an SSL certificate unlocks all of them without any changes to the application codebase.